

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1711

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour

Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I D.Balaji(192512368) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student : D.Balaji

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the next path based only on the heuristic value $h(n)$, which estimates the straight-line distance from the current location to the destination. It always chooses the node that appears to be closest to the goal without considering the total travel cost.

Behavior in this Scenario

- The drone always chooses the route that appears closest to the destination.
- It ignores the actual travel cost caused by bad weather or difficult terrain.
- If a selected road becomes blocked, the drone must search again from its current position.
- It can quickly find a route but may not always find the safest or shortest path.

Strengths

- Fast decision-making.
- Less memory usage.
- Suitable for emergency situations requiring quick responses.
- Simple to implement.

Weaknesses

- Does not guarantee the optimal path.
- Can choose dangerous or expensive routes.
- Performance decreases when the heuristic is inaccurate.
- Frequently needs re-planning in dynamic environments.

*ii. Explain how A would handle the same situation, including how it balances cost and heuristic**

A Search Algorithm*

A* Search uses the evaluation function:

$$f(n) = g(n) + h(n)$$

Where:

- **$g(n)$** = Actual cost from the start node to the current node.
- **$h(n)$** = Estimated cost from the current node to the goal.
- **$f(n)$** = Total estimated cost.

Behavior in this Scenario

The drone evaluates both the distance already travelled and the estimated distance remaining. If heavy rain, damaged roads, or unsafe terrain increase the travel cost, A* automatically considers these factors and chooses a safer and more economical route.

Advantages

- Finds the optimal path when the heuristic is admissible.
- Balances speed and path quality.
- Reduces unnecessary exploration.
- Provides safer navigation in changing environments.

Disadvantages

- Requires more memory than Greedy Search.
- Computationally more expensive.
- Needs updated information whenever the environment changes.

iii. Analyze the impact of heuristic accuracy on both algorithms

The heuristic function greatly affects the performance of both algorithms.

Accurate Heuristic

- Greedy Search quickly reaches the destination.
- A* finds the optimal path efficiently.
- Fewer unnecessary nodes are explored.

Inaccurate Heuristic

- Greedy Search may move toward blocked or unsafe paths.

- A* still finds a good solution but explores more nodes.
- Execution time and computational cost increase.

Comparison

Greedy Best-First Search	A* Search
Highly dependent on heuristic accuracy	Less affected because it also considers actual cost
May produce poor routes	Usually finds the best route
Faster	Slightly slower but more reliable

iv. Discuss how dynamic changes (blocked paths) affect search performance

In disaster environments, roads may become blocked suddenly due to floods or landslides.

Effect on Greedy Search

- May continue toward blocked roads.
- Requires complete re-planning.
- Wastes time and battery power.

Effect on A*

- Updates path costs using new information.
- Recalculates a better alternative route.
- More adaptable to changing environments.
- Maintains safer navigation.

Therefore, dynamic changes reduce the efficiency of both algorithms, but A* adapts better than Greedy Search.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

*Recommended Algorithm: A Search**

Justification

- Produces optimal routes by considering both actual and estimated costs.
- Avoids dangerous and expensive paths.
- Performs well even when road conditions change.
- Provides better safety for emergency medicine delivery.
- Reduces overall delivery delay.
- Ensures reliable decision-making in uncertain environments.

Although Greedy Best-First Search is faster, it may choose risky or blocked routes. Therefore, *A Search is the most suitable algorithm for AI-powered emergency drone navigation because it offers the best balance between speed, optimality, and safety.* *

Conclusion

For emergency medicine delivery in flood-affected areas, A* Search is more effective than Greedy Best-First Search because it considers both actual travel cost and heuristic information. This enables the drone to choose safer, shorter, and more reliable routes while adapting to dynamic environmental changes.

QUESTION 2

Scenario

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The main objective is to minimize total waiting time, fuel consumption, and traffic congestion. The system must continuously adjust traffic signals based on real-time traffic flow. However, the search space is extremely large, traffic conditions change dynamically, and the system may get stuck in locally optimal solutions.

Formulate this as an Optimization Problem and Explain Why Traditional Search is Inefficient

Optimization Problem

The traffic signal optimization problem can be formulated as an optimization problem where the AI system aims to find the best signal timings that minimize traffic congestion and maximize traffic flow.

Objective Function

The AI system should minimize:

- Total waiting time of vehicles.
- Fuel consumption caused by idling vehicles.
- Traffic congestion at intersections.
- Overall travel time.

Decision Variables

- Green signal duration.
- Red signal duration.
- Yellow signal duration.
- Signal sequence at each intersection.

Constraints

- Minimum and maximum signal timing limits.
- Emergency vehicle priority.
- Pedestrian crossing time.
- Safety regulations.
- Synchronization between nearby intersections.

Why Traditional Search is Inefficient

Traditional search algorithms like BFS and DFS are not suitable because:

- The number of possible signal combinations is extremely large.
- Traffic conditions change every second.
- Exploring every possible solution requires excessive time.
- Real-time decision-making is impossible using exhaustive search.

Therefore, optimization algorithms are more suitable for solving this problem.

i. Explain how Hill Climbing would work in this scenario and identify its limitations

Hill Climbing Algorithm

Hill Climbing is a local search algorithm that continuously improves the current solution by moving to a neighboring solution with a better evaluation score.

Working in Traffic Signal Optimization

1. Start with an initial traffic signal timing.
2. Evaluate the traffic performance.
3. Modify one or more signal timings slightly.
4. Compare the new solution with the current solution.
5. If the new solution reduces waiting time and congestion, accept it.
6. Repeat the process until no better neighboring solution exists.

Advantages

- Simple to implement.
- Fast execution.
- Requires less memory.
- Produces quick improvements.

Limitations

- Can become trapped in local maxima.
- Cannot guarantee the global optimal solution.
- Performance depends heavily on the starting solution.
- May stop even when better solutions exist elsewhere.

ii. Discuss issues like Local Maxima, Plateaus, and Ridges with Real-World Interpretation

1. Local Maxima

A local maxima is a solution that appears better than all nearby solutions but is not the best overall solution.

Traffic Example

One intersection has very smooth traffic flow, but nearby intersections experience heavy congestion. The AI incorrectly assumes this is the best overall solution.

2. Plateau

A plateau is a flat region where neighboring solutions have nearly identical evaluation values.

Traffic Example

Changing signal timings by a few seconds produces no noticeable improvement in traffic flow. The algorithm cannot determine which direction to move.

3. Ridge

A ridge occurs when the optimal solution requires several coordinated changes instead of a single change.

Traffic Example

Improving traffic requires adjusting multiple intersections simultaneously. Changing only one traffic signal provides little or no improvement.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

Simulated Annealing

Simulated Annealing is an advanced optimization algorithm that occasionally accepts worse solutions to escape local maxima.

How It Helps

- Escapes local maxima.
- Explores different regions of the search space.

- Gradually improves toward a better solution.
- Works well in dynamic traffic conditions.

Advantages

- Better exploration.
- Higher probability of finding the global optimum.
- Reduces chances of getting stuck.

Genetic Algorithm

A Genetic Algorithm is inspired by natural evolution.

Working Process

1. Generate multiple traffic signal timing solutions.
2. Evaluate each solution.
3. Select the best-performing solutions.
4. Apply crossover to combine good features.
5. Apply mutation to introduce new possibilities.
6. Repeat until an optimal solution is obtained.

Advantages

- Searches many solutions simultaneously.
- Avoids local maxima.
- Produces high-quality solutions.
- Suitable for large and complex optimization problems.

iv. Recommend the Best Approach and Justify Your Choice Based on Scalability and Adaptability

Recommended Algorithm: Genetic Algorithm

Justification

- Handles a very large search space efficiently.
- Optimizes multiple objectives simultaneously.
- Continuously adapts to changing traffic conditions.
- Produces high-quality signal timing solutions.
- Easily scales to hundreds of intersections.

- Avoids local maxima through crossover and mutation.
- Improves overall traffic flow, reduces waiting time, and lowers fuel consumption.

Although Hill Climbing is fast, it often becomes trapped in local maxima. Simulated Annealing improves this limitation, but the **Genetic Algorithm** is the most suitable approach because it offers better scalability, adaptability, and optimization performance for large smart-city traffic systems.

Conclusion

Traffic signal optimization is a complex optimization problem with a huge search space and continuously changing traffic patterns. Traditional search algorithms are inefficient because they cannot handle such complexity in real time. Hill Climbing provides quick improvements but suffers from local maxima. Simulated Annealing overcomes some of these issues, while the **Genetic Algorithm** provides the best balance of scalability, adaptability, and solution quality, making it the most effective choice for smart city traffic management.

QUESTION 3

Scenario

An autonomous rover is deployed on Mars to explore unknown terrain. The rover must navigate safely to collect samples, avoid hazards such as craters and rocks, operate with limited sensing capability, and make decisions without a complete map. As it explores, it continuously updates its knowledge, making this a real-time decision-making problem.

i. Explain why this problem requires an Online Search Agent instead of Offline Search

Online Search Agent

An **Online Search Agent** makes decisions while exploring the environment. It does not require complete knowledge of the map before starting its task. Instead, it gathers information continuously and updates its decisions based on newly discovered data.

Why Online Search is Required

- The Mars rover does not have a complete map of the terrain.
- New obstacles such as rocks and craters are discovered during exploration.
- The rover must make immediate decisions while moving.
- It continuously updates its path using newly collected information.
- Online search enables the rover to adapt to unknown and changing environments.

Why Offline Search is Not Suitable

Offline search assumes that the complete environment is known before the search begins. Since the Mars rover explores an unknown planet, offline search cannot provide accurate or safe navigation.

ii. Analyze the Challenges of Partial Observability and Dynamic Environments

1. Partial Observability

Partial observability means the rover can only sense a limited area around its current location.

Challenges

- Hidden obstacles cannot be detected in advance.
- The rover has incomplete information about the environment.
- Path planning becomes more difficult.
- Unexpected hazards may appear while moving.

2. Dynamic Environment

A dynamic environment changes over time while the rover is operating.

Challenges

- New obstacles may appear unexpectedly.
- Terrain conditions may change.
- Communication delays with Earth prevent immediate human assistance.
- The rover must quickly modify its route without stopping its mission.

iii. Describe how the Rover builds and updates its Internal Model

The rover maintains an **internal model** of the environment based on information collected by its sensors.

Working Process

1. Start exploring the unknown terrain.
2. Collect sensor data about nearby rocks, craters, and safe paths.
3. Store the discovered information in memory.
4. Update the internal map continuously.
5. Recalculate the safest path whenever new obstacles are detected.
6. Repeat the process until the mission is completed.

Advantages

- Improves navigation accuracy.
- Helps avoid dangerous areas.
- Supports better future decision-making.
- Reduces unnecessary movement and saves energy.

iv. Compare Online Search with Uninformed and Informed Search Strategies

Feature	Online Search	Uninformed Search	Informed Search
Environment Knowledge	Unknown	Complete	Complete
Uses Heuristic	No	No	Yes
Decision Making	During exploration	Before exploration	Before exploration
Adaptability	High	Low	Medium
Suitable for Mars Rover	Yes	No	Limited

Comparison

- **Online Search** continuously learns and updates while exploring.
- **Uninformed Search** (such as BFS or DFS) explores without heuristic information and assumes the environment is known.
- **Informed Search** (such as A*) uses heuristic information but still requires a map of the environment.
- Therefore, **Online Search** is the most suitable approach for unknown environments like Mars.

v. Justify the Most Suitable Approach Considering Uncertainty, Adaptability, and Safety

Recommended Approach: Online Search Agent

Justification

- Works effectively in unknown environments.
- Continuously learns from sensor data.
- Quickly adapts to newly discovered obstacles.
- Ensures safe navigation by avoiding hazards.
- Supports real-time decision-making.
- Reduces mission risk and improves exploration efficiency.
- Does not require a complete map before starting the mission.

Because Mars exploration involves uncertainty, limited sensing, and continuously changing information, an **Online Search Agent** provides the best balance of adaptability, safety, and intelligent decision-making.

Conclusion

Mars rover exploration is a real-time AI problem because the environment is unknown and only partially observable. The rover must continuously collect information, update its internal model, and make intelligent navigation decisions while avoiding hazards. Therefore, an **Online Search Agent** is the most suitable approach as it provides high adaptability, improves safety, and enables efficient exploration in uncertain environments.

4)Scenario

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints such as no student should have overlapping exams, limited exam halls are available, certain subjects must be scheduled before others, faculty availability must be considered, and special accommodations should be provided for some students. The complexity increases as the number of students and courses grows.

Formulate the Problem as a Constraint Satisfaction Problem (CSP)

A **Constraint Satisfaction Problem (CSP)** is an AI problem where a solution is found by assigning values to variables while satisfying all given constraints.

For exam timetable generation:

- **Variables:** Subjects/Courses
- **Domains:** Available exam dates, time slots, and exam halls
- **Constraints:** No overlapping exams, hall capacity, faculty availability, subject order, and special accommodations.

The objective is to generate a valid exam timetable that satisfies all constraints.

i. Clearly Define Variables, Domains, and Constraints with Explanation

1. Variables

Variables are the subjects or courses that need to be scheduled.

Example:

- Mathematics
- Artificial Intelligence
- Database Management System
- Operating Systems
- Computer Networks

Each subject is treated as one variable.

2. Domains

A domain is the set of possible values that can be assigned to each variable.

Example Domains

- Time Slot 1 (9:00 AM – 12:00 PM)
- Time Slot 2 (2:00 PM – 5:00 PM)
- Monday
- Tuesday
- Wednesday
- Hall A
- Hall B
- Hall C

Each subject can be assigned to one available day, time slot, and exam hall.

3. Constraints

The timetable must satisfy the following constraints:

Hard Constraints

- No student should have two exams at the same time.
- One exam hall cannot host two exams simultaneously.
- Hall capacity should not be exceeded.
- Faculty members must be available during the exam.
- Certain subjects must be conducted before other subjects.
- Special accommodation students should receive appropriate seating arrangements.

Soft Constraints

- Reduce long gaps between exams.
- Avoid scheduling many difficult subjects on consecutive days.
- Balance faculty workload.
- Improve student convenience.

A valid timetable must satisfy all hard constraints and should satisfy as many soft constraints as possible.

ii. Explain How Backtracking Search Works in this Scenario

Backtracking Search Algorithm

Backtracking is a search algorithm that assigns values one by one. If any constraint is violated, it returns to the previous assignment and tries another possible value.

Working Steps

1. Select the first subject.
2. Assign an available date, time slot, and hall.
3. Check whether all constraints are satisfied.
4. If the assignment is valid, move to the next subject.
5. If any constraint is violated, undo the assignment (backtrack).
6. Try another available time slot or hall.
7. Repeat until all subjects are scheduled successfully.

Advantages

- Guarantees a valid solution if one exists.
- Easy to implement.
- Eliminates invalid schedules during the search.
- Suitable for CSP problems.

Limitations

- Becomes slow for very large timetables.
- Requires many backtracking steps when constraints increase.

iii. Discuss How Constraint Propagation (Forward Checking) Improves Efficiency

Forward Checking

Forward Checking is a constraint propagation technique that removes invalid choices before they are selected.

Working Process

1. Assign a time slot to one subject.
2. Remove that same time slot from conflicting subjects.
3. Continue reducing possible choices after every assignment.
4. Detect conflicts early.
5. Prevent unnecessary backtracking.

Advantages

- Reduces the search space.
- Detects impossible schedules early.
- Improves execution speed.
- Minimizes unnecessary backtracking.
- Produces solutions more efficiently.

Example

If **Artificial Intelligence** is scheduled on **Monday Morning**, then all subjects having the same students cannot use **Monday Morning**. Those options are removed immediately from their domains.

iv. Analyze Real-World Challenges and Justify the Best Strategy for Scalability

Real-World Challenges

- Large number of students and courses.
- Limited exam halls.
- Faculty availability changes.
- Last-minute timetable modifications.
- Special accommodation requirements.
- Avoiding student exam clashes.
- Managing multiple departments simultaneously.

Recommended Strategy

Backtracking + Forward Checking

Justification

- Efficiently handles multiple constraints.
- Reduces unnecessary search using constraint propagation.
- Produces valid timetables with fewer conflicts.
- Easily scales to universities with thousands of students.
- Adapts well to timetable changes.
- Improves scheduling speed and accuracy.

This combination provides a practical and scalable solution for automatic exam timetable generation.

Conclusion

Exam timetable generation is a **Constraint Satisfaction Problem (CSP)** because multiple variables must be assigned values while satisfying several constraints. Backtracking Search systematically finds valid schedules, while Forward Checking improves efficiency by eliminating invalid choices early. Therefore, **Backtracking with Forward Checking** is the most suitable strategy for generating accurate, conflict-free, and scalable university exam timetables.

5)Scenario

A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, make decisions within strict time limits, evaluate complex game states with multiple possible moves, and handle an extremely large decision tree where computation is expensive.

i. Explain How the Minimax Algorithm Models this Problem

Minimax Algorithm

The **Minimax Algorithm** is an adversarial search algorithm used in two-player games. It assumes that one player (the AI) tries to maximize its score, while the opponent tries to minimize the AI's score.

Working Process

1. The AI generates all possible legal moves.
2. It predicts the opponent's possible responses.
3. A game tree is created containing future game states.
4. The AI evaluates each possible outcome using an evaluation function.
5. The AI chooses the move with the highest Minimax value.
6. This process continues until the best possible move is selected.

Example

Suppose the AI has three possible moves:

- **Move A → Score = 8**
- **Move B → Score = 6**
- **Move C → Score = 4**

The opponent always tries to reduce the AI's score. The AI selects the move that provides the **best guaranteed outcome**, even if the opponent plays optimally.

Advantages

- Makes intelligent decisions.
- Considers future consequences of each move.
- Guarantees the best move when the game tree is fully explored.
- Widely used in board games such as Chess, Tic-Tac-Toe, and Checkers.

ii. Analyze the Computational Challenges of Large Game Trees

In real-time strategy games, every move creates many new possible game states.

Challenges

- Huge number of possible moves.
- Large search space.
- High memory consumption.
- Increased computation time.
- Difficult to make decisions within strict time limits.
- Complex evaluation of future game states.

Example

If each move has **10 possible actions** and the AI searches **6 levels deep**, the total number of nodes becomes:

$$10^6 = 1,000,000 \text{ nodes}$$

Evaluating such a large number of nodes is computationally expensive and slows down decision-making.

iii. Explain How Alpha-Beta Pruning Improves Efficiency with Detailed Reasoning

Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization technique used with the Minimax algorithm. It reduces the number of nodes evaluated by eliminating branches that cannot influence the final decision.

Working Process

- **Alpha (α):** Best value found so far for the maximizing player.
- **Beta (β):** Best value found so far for the minimizing player.
- If $\alpha \geq \beta$, the remaining branches are ignored because they cannot produce a better result.

Advantages

- Reduces unnecessary node evaluation.
- Speeds up decision-making.

- Produces the same optimal result as Minimax.
- Saves memory and computation time.
- Enables deeper search within the same time limit.

Example

Without Alpha-Beta Pruning:

- Nodes evaluated = **1000**

With Alpha-Beta Pruning:

- Nodes evaluated = **400**

The AI reaches the same optimal decision while performing much less computation.

iv. Design an Evaluation Function and Justify the Factors Considered

Evaluation Function

An evaluation function assigns a numerical score to each game state.

Example Evaluation Function

Evaluation Score = (Attack Power + Resources + Health + Territory Control) – (Enemy Strength + Damage Taken)

Factors Considered

- 1. Attack Power**
 - Stronger attacking units receive higher scores.
- 2. Available Resources**
 - More resources allow building stronger units.
- 3. Health of Units**
 - Healthier units increase the chance of winning.
- 4. Territory Control**
 - Controlling more strategic locations gives an advantage.
- 5. Enemy Strength**
 - Stronger enemies reduce the evaluation score.
- 6. Damage Taken**
 - Greater damage lowers the AI's chances of winning.

Why These Factors?

These factors help the AI estimate the overall strength of its current position and choose moves that maximize the probability of victory.

v. Recommend a Strategy for Balancing Optimality and Real-Time Performance

Recommended Strategy: Minimax with Alpha-Beta Pruning

Justification

- Produces optimal decisions using the Minimax algorithm.
- Eliminates unnecessary branches through Alpha-Beta Pruning.
- Reduces computation time significantly.
- Supports deeper game-tree exploration.
- Suitable for real-time games with strict time limits.
- Improves AI performance against human players.
- Maintains a balance between decision quality and execution speed.

This combination enables the AI to make intelligent and fast decisions without evaluating every possible game state.

Conclusion

Real-time strategy games require the AI to make quick and intelligent decisions in extremely large search spaces. The **Minimax algorithm** provides optimal decision-making by considering both the AI's and the opponent's moves. **Alpha-Beta Pruning** enhances Minimax by eliminating unnecessary branches, reducing computation time while maintaining the same optimal result. Therefore, **Minimax with Alpha-Beta Pruning** is the most suitable approach for developing efficient and competitive game AI.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand